

在DeepSearch中用DeepSeek-R1来做动作决策会更好么？

原创 Jina AI Jina AI 2025年04月02日 10:05 北京



众所周知，DeepSeek R1 这种模型在推理任务上很能打，尤其是在数学和编程这些逻辑性强的领域。那么我们能直接把这种强大的推理能力搬到 DeepSearch 这种需要动态规划、多轮交互的深度搜索场景里吗？

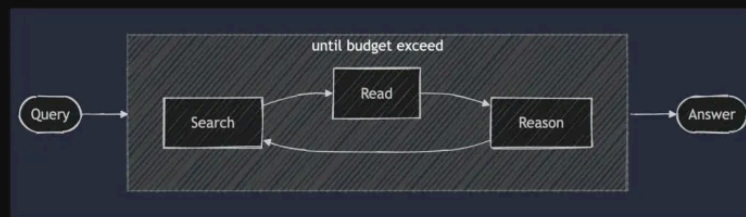
换句话说，一个精于逻辑推演的模型，真的能驾驭好在网上深挖信息时所需的那种模糊性判断和策略切换吗？还是说，它的推理天赋其实被“锁死”在了它熟悉的训练领域里？

在这篇文章中，我们让 R1 直面这种复杂的深度搜索(Deep Search)挑战，看看它的表现如何。

在我们看来，DeepSearch 本质上就是一个大的 while 循环。给定一个搜索任务和一定的资源限制(token 消耗的上限)，它会不断地重复“搜索”、“阅读”、“思考”这几个动作，直到找到它认为最好的答案为止。

在这个循环里，LLM 的一个核心任务就是根据当前的记忆状态，来决定下一步该采取什么行动：是接着搜呢？还是去读某个网页？或者干脆直接给出答案？

深度搜索的实现范式之一：WHILE LOOP



搜

获取搜索引擎结果页
使用s.jina.ai API

读

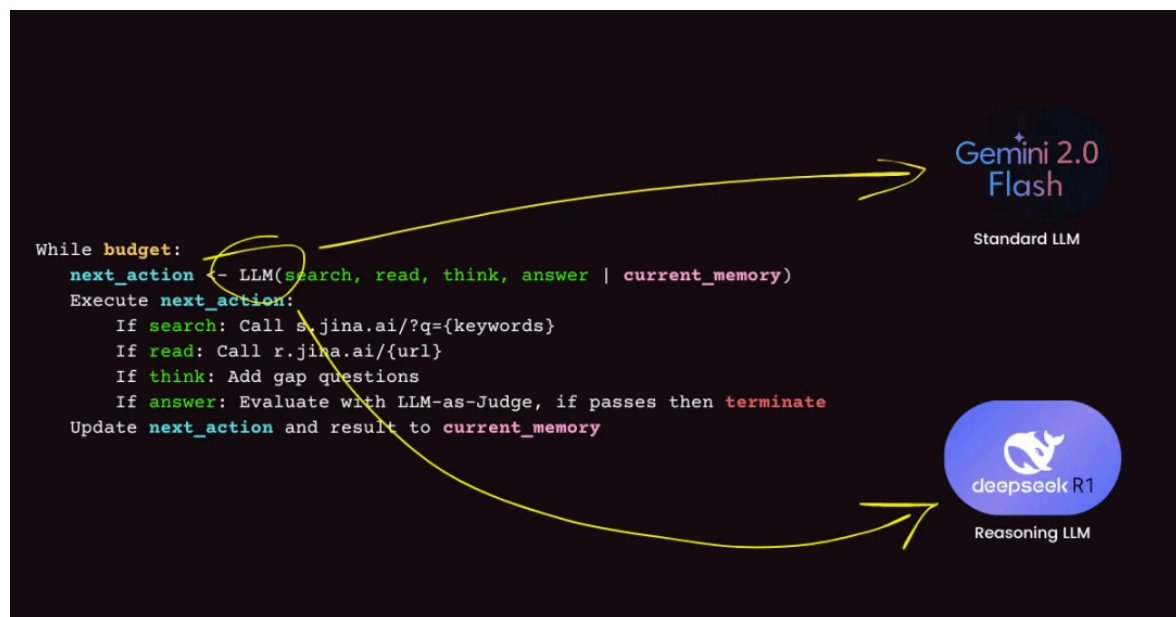
读取网页内容
使用r.jina.ai API

思

下一步干嘛？继续搜、读还是作答？
使用gemini

在我们 Jina AI DeepSearch (search.jina.ai) 的开源实现里，决定下一步动作这个环节，用的是一个标准的 LLM (**gemini-2.0-flash**)。为什么说它“标准”呢？因为它的决策逻辑，是我们通过精心设计的提示词给明确规定好的，工作方式就是传统大模型的“输入一段指令，输出一段结果”，这个结果里包含了用 JSON 格式 (Structured output) 指定的下一步行为和动作。

所以我们很自然的问自己：既然这个决策步骤这么关键，那如果我们不用这种标准的 LLM，而是换上一个专门用来做推理的模型，DeepSearch 的整体效果会不会更好呢？



这篇文章里，我们就来试试把原来用 **gemini-2.0-flash** 来做决策的地方，换成 DeepSeek R1 671B 这个专攻推理的 LLM。直觉上来讲，R1 拥有更强的推理能力，应该能帮助 DeepSearch 更好地解决在网上搜索信息、分析搜索结果时遇到的复杂问题，并选择最有效的动作。

为了检验这个想法，我们就拿一个实际的任务来试试水：让它规划一个详细的三天假期行程，然后看看效果到底怎么样。

搜索 Agent 在做这种深度搜索任务时，碰到的问题跟咱们人类平时查资料也差不多。所以在这次旅行规划的任务里，模型很可能会踩到下面这些坑：

1. **知识有缺口（有些信息得依赖其他信息才能获得）**：比如，你想去埃菲尔铁塔，但不知道法定节假日开不开门。那你不仅得查到铁塔的节假日开放时间，还得搞清楚法国的法定节假日是哪几天才行。
2. **信息错误或者过时了**：比如，一篇 2020 年的旅游攻略说罗马有家餐厅周日也开门，结果你兴冲冲跑过去，发现人家早就改了营业时间，现在周日不营业了。
3. **信息相互矛盾**：比如，一个旅游网站说纽约某家酒店是包早餐的，但另一个网站又说房费里不含早餐，你和模型都一头雾水。
4. **信息模糊**：比如，一个旅游论坛帖子里提到“巴塞罗那附近有个超漂亮的海滩”，但既没说是哪个海滩，也没给具体路线，让你很难定位到那个地方。

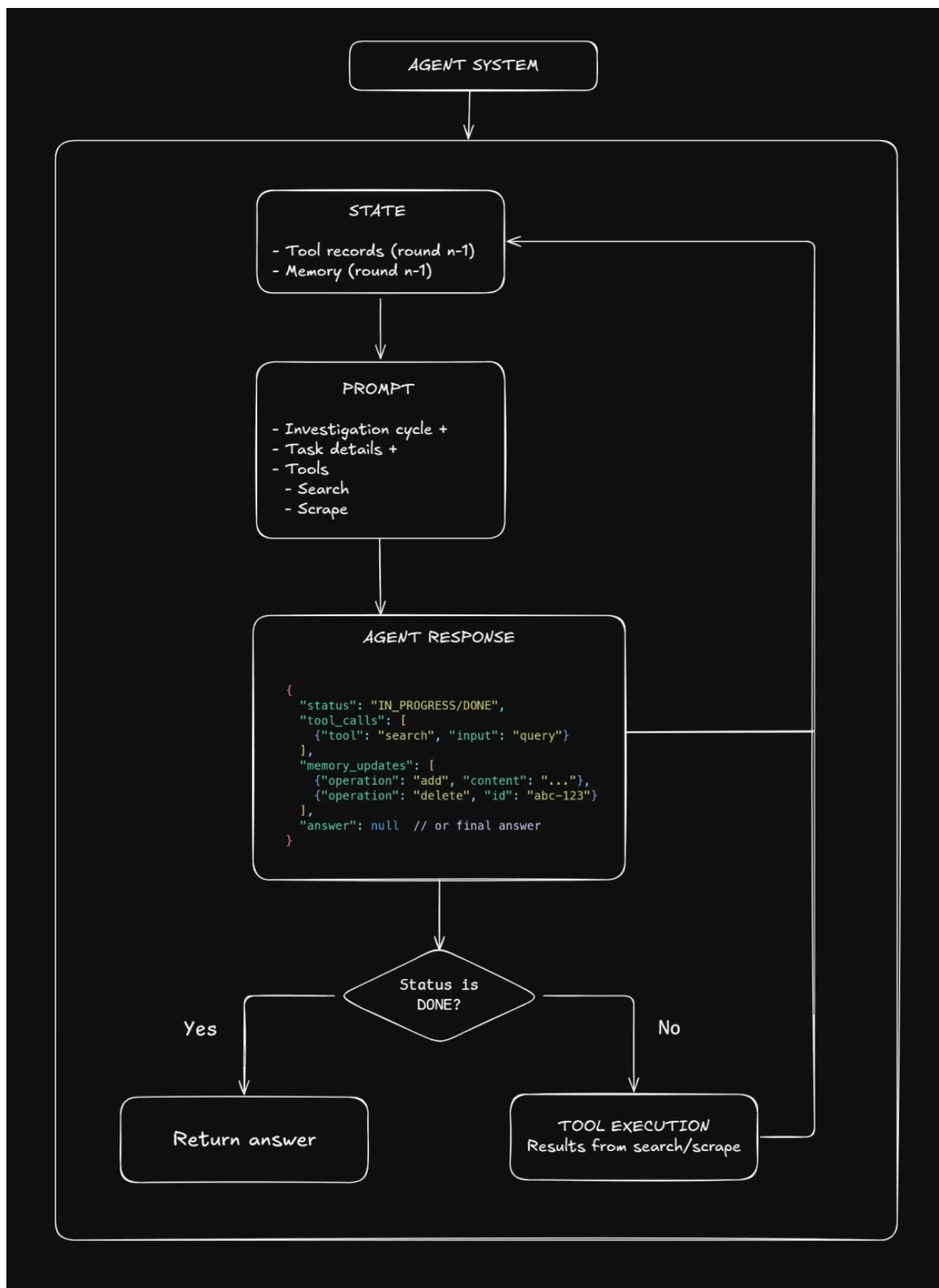
R1 能把复杂的任务拆解成一步步可执行的操作，也能识别信息里的缺口和矛盾，还能绕开像网站打不开、需要付费订阅才能看这种障碍。它具备足够的推理能力来收集所需信息，并最终整合出答案。

不过光靠 R1 自己，还不足以独立完成规划旅行计划，这件事还需要真正去搜索信息，并且理解搜回来的结果才行。因此我们必须把它放进一个我们设计的框架里，再给它的能力升个级。

大循环的实现

代码已在Google Colab上开源：https://colab.research.google.com/drive/18sqU8_eWqFleKqpd-SnGDNmZ_P1KLfXw?usp=sharing#scrollTo=2jFWdbnp_6ws

虽然 R1 是我们这个搜索 Agent 的核心引擎，但系统的完整运行还需要集成工具、一个用来记录各种信息的状态，以及一个内容相当庞大的提示词。其简化架构如下：



- **工具 (Tools)** : 模型可以随时调用这些工具去网上搜索信息、抓取网页内容。搜到的结果会存到状态里。
- **状态 (State)** : 负责记录工具的执行结果、任务进行到哪一步了 (也就是任务状态), 以及已经获取的知识。这个状态信息是直接维护在提示词内部的。
- **提示词 (Prompt)** : 一个非常非常大的提示词, 里面包含了详细的指令, 定义模型任务的目标、该怎么一步步做, 同时也负责存储上面提到的状态信息。

接下来我们会详细讲讲这些组件，特别是提示词的设计。但简单来说，该系统的工作流程如下：

一开始，我们给模型一个提示词，里面的“状态”部分是空的。只要搜索任务还在进行，这个搜索 Agent 就会不停地重复以下循环，直到它最终给出答案为止：

1. **思考**：模型会先解析提示词中的任务要求和当前状态，推理出接下来该怎么用好手上的工具才能最高效地找到答案的搜索策略。
2. **决策输出**：模型会输出一个 JSON 对象，里面明确指定当前任务状态（是“进行中”还是“已完成”）、需要更新的记忆内容、要调用的工具以及当前的答案（如果还没找到，就是 null）。
3. **工具执行与状态更新**：Agent 根据模型的指令，去异步调用相应的工具。工具返回的结果，连同第 2 步生成的 JSON 对象，都塞回到提示词里。
4. **再次循环**：更新后的提示词会被重新输入给模型，启动新一轮“思考 -> 决策 -> 执行”循环。

一旦模型在输出里给出了答案（即答案字段不再为 null），任务就算完成，答案也就交差了。

💡 小提示：要想更直观地了解这个 Agent 是怎么工作的，我们建议你亲自跑一下原 notebook 文件的代码，看看每一次迭代都输出了些什么。

好了，大概了解了整体流程之后，我们再来依次看看工具、状态和提示词这几个部分：

工具调用

因为 R1 自己不会上网搜索或者抓取网页内容，所以我们接入了 Jina Reader API 来给它扩展能力。这个 API 主要有两种模式：

1. **搜索模式 (s.jina.ai)**：根据相关的关键词在网上搜索，返回搜索引擎的结果（即SERP页，其包含了每个结果的网址、标题和描述）。
2. **阅读模式 (r.jina.ai)**：抓取URL背后的网页内容，清理后以 Markdown 格式返回。

考虑到 R1 有限的上下文窗口限制，我们无法将抓取的整个网页内容直接填充到提示词的工具结果 (Tool Results) 部分。所以，我们还需要额外的工具，在把信息喂给模型之前，先把最相关的部分筛选出来：

1. LangChain 的递归字符文本分割器 (Recursive Character Text Splitter)：我们使用 LangChain 提供的工具把长篇内容切成小段，它会递归地依据段落、句子来切分，直到切成我们想要的大小。这样就能保证输出的内容更容易被 R1 有限的上下文窗口消化掉。
2. Jina Reranker：我们用 `jina-reranker-v2-base-multilingual` 模型，对切分好的文本片段进行相关性重排序 (rerank)，将排名最高的若干个片段合并，整合成最终提供给模型的结果。

可惜的是，DeepSeek R1 在使用工具调用这方面，跟OpenAI `o3-mini` 的方式不太一样。比如，用 `o3-mini` 的话，我们可以像下面这样写代码（直接在 API 请求里定义工具和参数）：

```
def scrape_page(url: str):
    """用 Jina Reader 读取网页"""

tools = [
    {
        "type": "function",
        "function": {
            "name": "scrape_page",
            "description": "Scrape the content of a webpage",
            "parameters": {
                "url": {"type": "string", "description": "The URL to scrape"}
            }
        }
    }
]

client = OpenAI()
response = client.completions.create(
    model="o3-mini",
    prompt=f"Scrape www.skyscanner.net/routes/gr/de/germany-to-crete.html",
    tools=tools
)
```

但 R1 就没这么方便了：它的 API 没有 tools 这个参数，也不会在响应里返回结构化的 tool_calls 信息。简单说，它就不是被训练来直接使用工具的（短期内估计也不会支持）。至少，不是传统意义上的那种工具调用支持。

不过嘛，我们还是想办法让 R1 输出 JSON 格式的工具调用指令，然后把工具调用的结果再喂给模型去分析。下面是让 R1 在提示词里输出工具调用的示例部分：

你必须返回一个包含以下内容的有效 JSON 对象：

```
{
  "tool_calls": [
    {"tool": "search", "input": "德国到克里特岛 2025年5月 最便宜航班"},
    {"tool": "scrape", "input": "<https://www.skyscanner.net/routes/gr/de/germany-to-crete.html"}
  ]
}
```

在第 n 次迭代中，模型输出了工具调用指令后，系统会执行这些工具调用，并将返回的结果嵌入到提示词的 Tool Results 部分，供模型在第 n+1 次迭代中进行推理。以下是工具结果的示例，显示德国-克里特岛航班的搜索结果：

Tool Results:

来源 1 : search: 德国到克里特岛 2025 年 5 月 最便宜航班

结果:

标题：便宜机票：德国 - 伊拉克利翁（克里特岛）(HER) | Eurowings

网址来源：<https://www.eurowings.com/en/booking/offers/flights-from/DE/to/GR/HER.html>

描述：德国到伊拉克利翁（克里特岛）的便宜机票 → Eurowings 带你飞往梦想目的地，票价低至 89.99 欧元*。

标题：你是人类还是机器人？

网址来源：<https://www.skyscanner.com/routes/fran/her/frankfurt-to-crete-heraklion.html>

描述：预订法兰克福到伊拉克利翁机场的单程机票 **\$78** 起，或往返机票仅 **\$154** 起。所示价格基于库存情况，可能会

Agent 的状态

状态负责跟踪任务的进展（用 Status 表示）和模型需要分析、更新的知识（用 Memory 表示）。简单说，它就是系统的工作记忆区和知识库。这部分信息存在提示词里一个叫做 `{{ workspace }}` 的区域，初始状态为空白：

```
Status: IN_PROGRESS
Memory:
... no memory blocks ..
```

随着模型对任务进行推理、运行工具并收集输出，状态里面就会填充进一些“记忆块”（memory blocks），这些记忆块是从工具的输出里提炼出来的，每个都有一个随机分配的唯一 ID。拿我们规划假期的例子来说，跑完一轮 Agent 迭代后，状态可能看起来像这样：

```
Status: IN_PROGRESS
Memory:
<nuz-032>可能的5月温暖目的地：马拉加（西班牙），克里特岛（希腊），阿尔加维（葡萄牙）</nuz-032>
<xwj-969>待抓取的克里特岛酒店详情网址：<https://www.tripadvisor.com/HotelsList-Crete-Beachf>
<vsc-583>待抓取的航班详情网址：<https://www.expedia.com/lp/flights/fra/her/frankfurt-to-her>
```

这些记忆块的更新是通过在模型的 JSON 响应里包含一个 `memory_updates` 列表来实现的：

```
{
  "memory_updates": [
    {"operation": "add", "content": "根据 Skyscanner 和 Iberia 的信息，2025年5月从柏林到特内"},
    {"operation": "delete", "id": "nuz-032"},
    ...
  ]
}
```

- **add** 操作用来添加一个新的记忆块，可以用来存一些重要信息，比如有价值的线索、发现的事实、需要填补的信息空白，以及已经做过的操作等等。
- **delete** 操作用来删除一个指定的记忆块，这样模型就能清理掉那些过时的、没用的或者错误的信息，保持工作区的整洁。

💡：补充一点，我们之前也试过 `replace`（替换）操作来更新信息，但发现模型倾向于生成大块大块的信息（过度依赖替换功能），所以最后决定把这个选项去掉了。

跟调用工具比起来，R1 在管理自己的记忆这方面就没那么熟练了。虽然 R1 模型是专门训练来解决复杂数学问题和编程任务的。这种训练让它能准确生成 JSON 对象、调用工具，但它并没有接受过管理类似记忆状态的训练（据我们所知，其他模型也没有专门做这方面的训练）。

用这种紧凑的、类似记忆的状态来存储信息，比起每一轮都把模型的全部输出存下来，有几个好处：**这种方法能压缩提示词里的信息量，防止上下文超出长度限制，同时也能让模型更专注于相关的知识点。我们选择用 JSON 格式来存，主要是因为它容易更新，不过在最终的提示词里，这些 JSON 会被渲染成人类更容易阅读的格式。**

即便如此，管理‘记忆’对 R1 来说还是有点超纲；所以我们不得不在提示词里加入好几条指令，一步步教模型怎么正确地操作这些记忆。下面就是我们提示词里负责处理这部分内容的指令：

... 提示词的其他内容 ...

记忆块使用指南 (Memory Block Usage)

- 每个记忆块都有一个唯一的 ID，格式是 `<abc-123>内容</abc-123>`
- 给不同的信息创建不同的记忆块：
 - * 发现的网址（包括已经探索过的和待处理的）
 - * 需要调查的信息缺口
 - * 已经采取的行动（避免重复）
 - * 有希望的将来探索线索
 - * 关键事实和发现
 - * 发现的矛盾或不一致之处
- 每个块聚焦一个想法或一条信息
- 从工具结果中记录信息时，务必注明来源
- 使用 ID 来跟踪和管理你的知识（例如，删除过时信息）
- 确保为你存储的事实和发现记录来源（网址）

线索管理 (Lead Management)

- 由于每轮最多只能调用 3 次工具，请将有希望的线索存起来以后用
- 为以后要抓取的网址创建专门的记忆块
- 为未来几轮可能要探索的搜索查询维护记忆块
- 根据与任务的相关性对线索进行优先级排序

... 提示词的其他内容 ...

完整的提示词

我们使用 Jinja 模板格式来创建提示词。它主要包含几个部分：

- 上下文 (Context)：在这个例子里就是当前的日期。
- 指令 (Instructions)：说明整个工作流程、告诉模型有哪些工具可用等等。
- 状态 (State)：就是我们前面讨论的‘记忆’状态。
- 工具输出 (Tool outputs)：来自搜索和网页抓取工具的结果。

下面是我们在深度搜索系统中使用的完整提示词，原文为英文，此处为中文翻译：

```
{% macro format_tool_results(tool_records) %}
{% for to in tool_records %}
来源 {{ loop.index }}: {{ to.tool }}: {{ to.input }}
结果:
{{ to.output }}
{% endfor %}
{% endmacro %}
```

日期: `{{ current\_date }}`.

你是一个信息分析和探索 Agent，通过系统性的调查来构建解决方案。

调查周期 (Investigation Cycle)

你按照一个持续的调查周期运作：

1. 查看当前的工作区（你的记忆块）
2. 分析新的工具结果（如果是第一轮，则分析初始任务）
3. 用新的见解更新记忆，并跟踪调查进展
4. 根据已识别的线索和信息缺口，决定接下来要调用哪些工具
5. 重复以上步骤，直到任务完成

记忆结构 (Memory Structure)

你的记忆在调查周期之间是持续存在的，包括：

- ****状态 (Status)****：总是在第一行，表明任务是“进行中 (IN_PROGRESS)”还是“已完成 (DONE)”
- ****记忆 (Memory)****：一系列离散的信息块，每个都有唯一的 ID

记忆块使用指南 (Memory Block Usage)

- 每个记忆块都有一个唯一的 ID，格式是 `- 给不同的信息创建不同的记忆块：
 - * 发现的网址（包括已经探索过的和待处理的）
 - * 需要调查的信息缺口
 - * 已经采取的行动（避免重复）
 - * 有希望的未来探索线索
 - * 关键事实和发现
 - * 发现的矛盾或不一致之处
- 每个块聚焦一个想法或一条信息
- 从工具结果中记录信息时，务必注明来源
- 使用 ID 来跟踪和管理你的知识（例如，删除过时信息）
- 确保为你存储的事实和发现记录来源（网址）

线索管理 (Lead Management)

- 由于每轮最多只能调用 3 次工具，请将有希望的线索存起来以后用
- 为以后要抓取的网址创建专门的记忆块
- 为未来几轮可能要探索的搜索查询维护记忆块
- 根据与任务的相关性对线索进行优先级排序

可用工具 (Available Tools)

- ****search****：用于对新主题或概念进行广泛的信息收集
 - * 示例：{"tool": "search", "input": "2023年可再生能源统计数据"}
- ****scrape****：用于从发现的网址中提取具体细节
 - * 示例：{"tool": "scrape", "input": "https://example.com/energy-report"}

工具使用准则 (Tool Usage Guidelines)

- ****何时使用 search****：遇到新概念、填补知识空白或探索新方向时
- ****何时使用 scrape****：发现的网址很可能包含详细信息时
- ****每轮最多调用 3 次工具****

- **绝不重复完全相同的工具调用**
- **务必将工具结果中的有价值信息记录在记忆块中**

响应格式 (Response Format)

你必须返回一个包含以下内容的有效 JSON 对象：

```
{
  "status_update": "IN_PROGRESS 或 DONE",
  "memory_updates": [
    {"operation": "add", "content": "新的见解或待调查的线索"},
    {"operation": "delete", "id": "abc-123"}
  ],
  "tool_calls": [
    {"tool": "search", "input": "具体的搜索查询"},
    {"tool": "scrape", "input": "https://discovered-url.com"}
  ],
  "answer": "当 status 为 DONE 时的最终、全面的答案"
}
```

重要规则 (Important Rules)

- "add" 操作会创建一个新的记忆块
你不需要指定 ID，系统会自动添加。
- "delete" 操作需要指定要移除的块的具体 ID
- 切勿虚构或编造信息 - 只能使用来自你记忆或工具结果的事实
- 切勿编造网址 - 只能使用通过工具结果发现的网址
- 关键：任何未记录在你记忆块中的信息将在下一轮丢失。例如，如果你发现一个可能要抓取的网页，你必须存储该网址
示例：{"operation": "add", "content": "发现相关网址：https://... 待抓取 ..."}
- 仅当你完全解决了任务时，才将 status 设置为 "DONE"
- 仅当 status 为 "DONE" 时，才包含 "answer" 字段

任务：

{{task}}

当前工作区：

{{workspace}}

工具结果：

{{ format_tool_results(tool_records) if tool_records else '... 没有之前的工具结果 ...' }}

重要：请按照上述格式生成一个有效的 JSON 响应。

请仔细思考：

- 你需要保留哪些信息
- 接下来要调用哪些工具
- 如何利用聚焦的记忆块系统地构建你的答案

💡 提示词开头那个 Jinja 宏函数（macro function）只是为了把工具返回的结果渲染得更规整一些。

效果评估

我们用前面的假期规划的例子来试试这个系统到底怎么样。下面是给到提示词的具体任务要求：

帮我规划一个欧洲的三天假期，时间在五月，预算不超过 2000 欧元。

1. 我需要具体的航班和酒店推荐。
 2. 我希望目的地天气暖和。
 3. 我希望酒店附近有海滩。
- 我住在德国。

跑了五轮循环之后，最终结果是这样的：

****克里特岛 3 日游计划 (总花费约 €600-800)****

 ***航班 (从法兰克福出发):***

- Skyscanner (天巡): €154 往返 (2025年5月日期)
来源: <<https://www.skyscanner.com/routes/fran/her/frankfurt-to-crete-heraklion.html>>
- 备选: Eurowings (欧洲之翼航空) €180 往返

 ***海滨酒店:***

1. ****Georgiopolis Beach Hotel**** (耶奥伊乌波利斯海滩酒店)
 - 直达海滩, 可租自行车 €10
 - 来源: <<https://www.tripadvisor.com/HotelsList-Crete-Beachfront-Cheap-Hotels-zfp13>>
2. ****JT Apartments**** (JT 公寓)
 - €57/晚, 评分 9.5/10, 步行 5 分钟到海滩
 - 来源: <<https://www.booking.com/region/gr/crete.html>>
3. ****FNK Apartments**** (FNK 公寓)
 - 海景房, 评分 9.6/10

 ***预算明细:***

- 机票: €154
- 酒店 (3 晚): €171-250
- 剩余预算: €1,600+ 用于餐饮/活动

注意: 所有酒店距离海滩均 <5 分钟。克里特岛五月平均气温 25°C (来源: TravelSupermarket)。

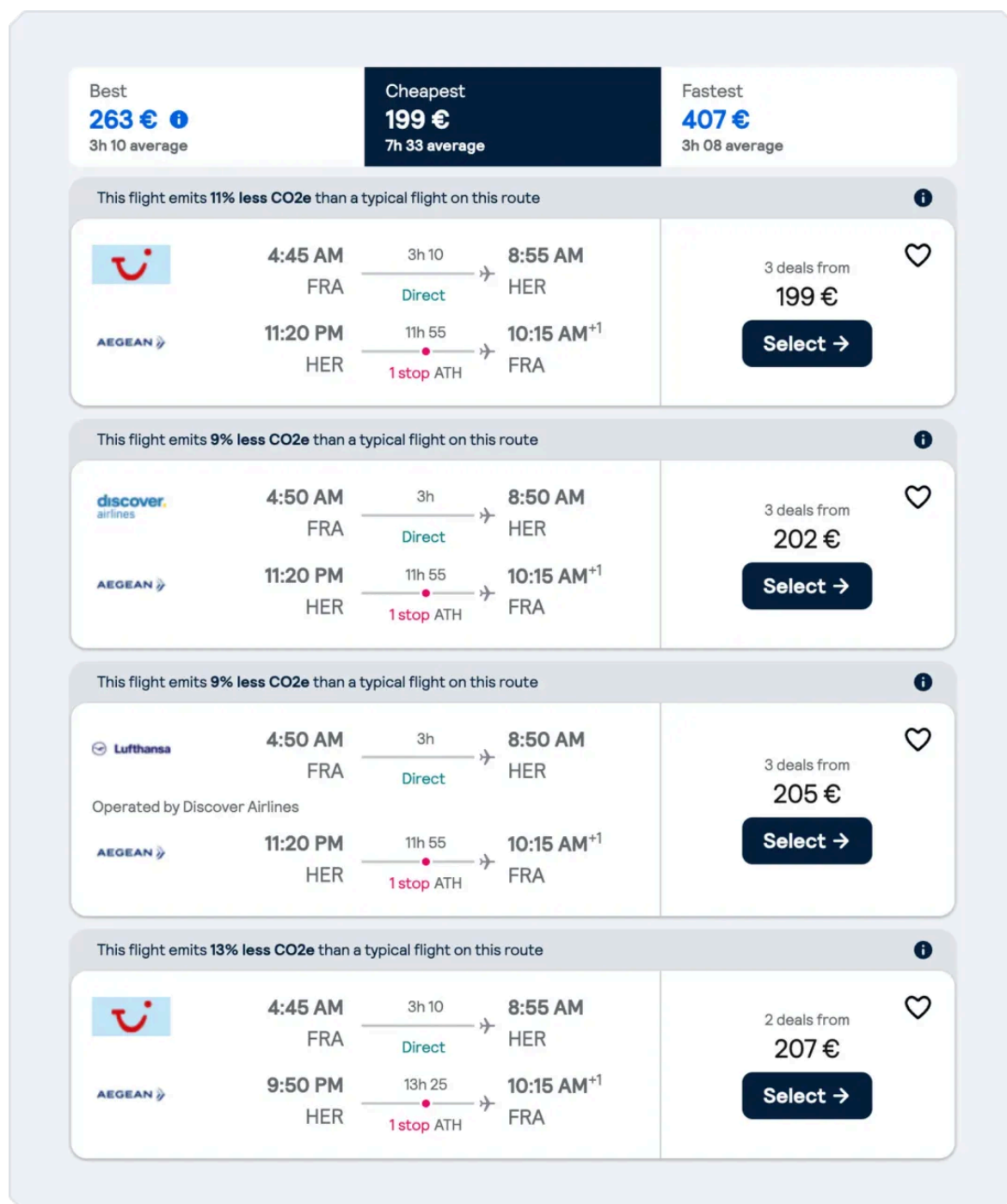
这答案看起来挺像回事儿的。但这真的靠谱吗？毕竟模型一本正经地胡说八道 (Hallucinate) 是出了名的，而且这个系统涉及到好几个部分协同工作，哪个环节出点错也是很有可能的。我们来仔细核对一下 R1 输出结果里的一些细节：

目的地和总预算

各项费用的计算总和是对的，毕竟 R1 是做过数学题训练的。至于每项费用准不准，我们后面再说。目的地也没毛病，克里特岛确实是个热门旅游地。

机票

机票价格这块儿就有点对不上了，我们来看看是哪里出了岔子。首先，这是 Skyscanner 上查到的 2025 年 5 月从法兰克福到伊拉克利翁的实际往返价格：



实际 Skyscanner 搜索法兰克福-伊拉克利翁 2025 年 5 月航班结果的截图

可以看到，价格都在 200 欧元左右，并不是它说的 154 欧元往返。那这错是哪儿来的呢？翻了翻日志，我们发现在第 3 轮加了个相关的记忆块：

```
{"operation": "add", "content": "克里特岛航班选项: Eurowings €89.99* 单程 ..."}

```

这个记忆块似乎是从附带的搜索结果里推断出来的：

来源 1：搜索：德国到克里特岛最便宜航班 2025年5月
 结果：
 ... 其他结果 ...
 标题：你是真人还是机器人？ (Are you a person or a robot?)

所以模型压根儿就没去抓取这个网页来确认结果, 不过就算抓了估计也没啥用。但不管怎么说, 它至少应该注意到, 这个搜索结果里根本没提到五月这个时间点啊。

酒店

酒店这块儿倒是没大错, 但我们觉得还有些可以改进的地方。首先, 我们希望模型能多花点心思把每家酒店的价格也找出来。它提供了其他信息, 但偏偏最重要的价格信息给漏了。

我们通过抓取那个用来生成酒店推荐的网址后得到的原始数据。发现它只显示了第一个和最后一个酒店的价格, 中间三个都跳过了。

其次, 我们还发现了一个问题。我们让 Reranker 固定只返回相关性最高的 5 个结果。但后来我们发现, 抓取下来的网页里, 相关的好酒店不止 5 家。我们本来可以不只看排名前 5, 而是去检查每一个结果的相关度分数, 把所有分数达标的结果都挑出来。

不过话说回来, 到底怎么设置 Reranker 效果最好 (比如是取固定数量, 还是按分数线来筛选), 并没有一个标准答案, 得看具体任务是什么。

所以, 最理想的办法是把整个抓取页面的全部内容都交给 R1 去分析。但遗憾的是, R1 能处理的文本长度 (上下文窗口) 有限, 根本塞不下这么多内容。

整体表现和待优化项

模型一开始表现还行, 但我们发现, **除非你明确指示它, 否则它很少会主动调整搜索策略或者制定出复杂的计划**。R1 在处理它拿手的数学和编程问题时, 很擅长这些, 但它没把这种能力迁移到搜索任务上来。当然, 我们也可以通过继续优化提示词, 或者用上好几个提示词组合来解决这个问题, 但这并非我们这次实验的主要目标。

我们还发现, **R1 在处理有‘时效性’的信息时做得不够好。简单来说, 如果搜索结果里没明确标出日期不对, 模型就默认这信息是对的, 也不去进一步核实**。比如, 在规划 5 月 1 号的航班时:

- 德国到克里特岛 \$80 **5 月 1 日**: **正确** – 模型可以相信这个信息。
- 德国到克里特岛 \$80 **1 月 1 日**: **错误** – 模型能正确识别并忽略掉这个。
- 德国到克里特岛 \$80 (**没写日期**): **误报 (False positive)** – 这种没写明日期的情况, 模型没去核实, 就错误地认为它是有效的。

如果我们要继续推进这个项目, 可能会考虑做下面这些改进:

- 跟踪记录“记忆块”的数量, 当状态信息存得太多的时候, 要求模型对这些条目做个总结。
- 要求模型在结束循环、给出最终答案之前, 把所有的线索都探索一遍, 确保没有遗漏。
- 更加强调要仔细核实那些有时效性的信息。
- 确保模型会通过抓取搜索工具返回的网址, 来二次核对搜索结果的准确性。

- 用未来那些支持更大上下文窗口的推理模型来测试我们的系统，不过这可能需要对提示词进行大量的重构和测试，才能适配新模型。

总结

自从 R1 模型发布以来（虽然也没多久），搜索领域就闹翻了天。现在训练推理模型的成本低得惊人（如 S1），为特定任务量身打造模型，门槛比之前更低了。我们基于 R1 做的这些尝试，正好为我们后续的探索打了个样：怎样才能更好地把各种工具跟侧重推理的 LLM 结合起来，去完成那些复杂的搜索任务。

回过头来看我们那个假期规划的例子。虽然结果看着有模有样，但也清清楚楚地暴露了 R1 的“偏科”问题：**它处理搜索和记忆任务的能力，跟它在数学、编程这些强项上的表现，差距还是挺大的。**

这个系统虽然成功地在 Token 预算范围内生成了个旅行计划，但在一些关键细节上，比如核实信息的时效性、彻底探索所有可能的选项等方面，它还是露了怯。这也恰恰暴露出：**推理模型在训练时所侧重的能力（数理逻辑），和我们试图让它应用的全新领域（需要大量信息检索和验证的任务）之间，还存在着**一道不小的鸿沟。这正是未来需要努力填补的地方。



DeepSearch 5

DeepSearch · 目录

[上一篇](#)

DeepSearch/DeepResearch 北京场分享干货回顾！本周日，上海再续！

[下一篇](#)

基于 LLM 的查询扩展：信息更全，搜索更准

